

Git / GitHub 入門

履歴を残し、共有するための最初の一步

kska6

2026 年 7 月 20 日

目次

1	Git とはなにか	3
1.1	バージョン管理とは	3
1.2	ディレクトリコピーによる管理との違い	3
1.3	Git のメリットとデメリット	4
2	Git と GitHub の違い	4
2.1	リモートリポジトリの選択肢	5
3	手元環境への導入	5
3.1	Git をインストールする	5
3.2	Git の初期設定	6
4	Git のローカル環境	6
5	リポジトリを作る	7
5.1	管理するファイルを選ぶ	7
5.2	練習用ディレクトリを用意する	7
5.3	最初のコミットを作る	8
5.4	README.md を作る	8
5.5	.gitignore を作る	8
5.6	README.md と .gitignore をコミットする	8
6	直線的なコミットを作る	9
6.1	README.md を追加で編集してコミットする	9
6.2	参考：変更の一部だけをステージする	10
6.3	参考：VS Code の Copilot でコミットメッセージ案を作る	11
7	GitHub の準備と認証	11
8	GitHub へ push する	12
8.1	GitHub の Web UI を使う	12

8.1.1	リモートを確認する	12
8.1.2	コミットを送る	12
8.2	GitHub CLI (gh) を使う	13
9	リモートリポジトリを clone する	13
9.1	git clone を使う	13
9.2	gh repo clone を使う	13
10	参考：作業中の変更を整理する	14
10.1	変更を取り消す	14
10.1.1	ワークツリーの未コミット変更を戻す	14
10.1.2	ステージを取り消す	14
10.1.3	コミットした変更を打ち消す	15
10.1.4	直前のコミットをやり直す	15
10.2	変更を一時退避する	16
11	ブランチを使う	16
11.1	作成と切り替え	16
11.2	ブランチをマージする	17
12	参考：履歴の節目にタグを付ける	17
12.1	注釈付きタグを作成して確認する	18
12.2	タグを GitHub へ送る	18
13	応用：Issue と Pull Request	18
13.1	Issue	19
13.2	Pull Request	19
14	応用：worktree で作業場所を分ける	19
14.1	追加の worktree を作る	20
14.2	活用例：LLM Agent の作業場所を分ける	20
14.3	worktree を片付ける	21
15	応用：リポジトリ構成を設計する	21
15.1	基本は1プロジェクト1リポジトリ	21
15.2	リポジトリの容量と管理対象	21
15.3	複数リポジトリをサブモジュールで組み合わせる	22
16	応用：Git の仕組み	22
16.1	コミットはスナップショットをつなぐ	23
16.2	保存内容から識別子を計算する	23
17	公式資料	23

この勉強会のゴール

この資料は、Git と GitHub を初めて使う人のための入門教材です。終了時には、次のことができる状態を目指します。

1. Git と GitHub の違いと、それぞれの役割を説明できる
2. 自分の PC に Git と GitHub CLI を導入し、初期設定と認証ができる
3. ファイルの変更をコミットとして記録し、履歴を確認し、ブランチで作業し、GitHub へ共有できる

コマンドの暗記よりも、「いま、どこからどこへ変更を動かしているか」を意識してください。

1 Git とはなにか

Git は、ファイルの変更履歴を記録する**バージョン管理システム**です。プログラムだけでなく、文章、設定ファイル、LaTeX 原稿など、テキストを中心としたファイルを管理できます。

履歴は自分の PC に保存されるため、インターネットに接続していなくても記録や確認ができます。

1.1 バージョン管理とは

バージョン管理は、ファイルの内容に加えて、いつ、誰が、何を、なぜ変更したかを履歴として残す仕組みです。

- 変更履歴を確認できる
- 過去の状態と現在の状態を比較できる
- 間違えた変更を取り消し、過去の状態を取り戻せる
- 複数の作業を分けて進め、あとで統合できる

Git はコミット時点のファイル構成を**スナップショット**として扱います。このスナップショットに作成者、日時、変更理由を表すメッセージなどを加えた確定記録が**コミット**です。作業の節目をコミットにすると、過去との差分や変更理由を、記憶ではなく履歴から確認できます。

1.2 ディレクトリコピーによる管理との違い

次のようなコピーも、簡単なバックアップとしては役立ちます。

```
report/  
report_20260719/  
report_final/  
report_final2/  
report_really_final/
```

しかし、ファイル名だけでは変更内容や保存理由が分かりません。コピーが増えるほど、最新版も見分けにくくなります。¹⁾

差分とは、二つの状態を比べたときに追加、削除、変更された行の内容を指します。

	ディレクトリコピー	Git
変更点	自分で比較する	差分として確認できる
変更理由	名前や別メモに残す	コミットメッセージに残す
過去の版	コピー単位で残す	コミット単位でたどる
並行作業	コピーが増えやすい	ブランチで分けられる

1.3 Git のメリットとデメリット

Git の主なメリットは次のとおりです。

- 変更履歴と差分を確認できるため、いつ、どこを変更したかをたどりやすい
- 履歴をローカルに保存するため、インターネットに接続していなくても高速に操作できる
- 元の履歴を保ったまま、ブランチで作業を分けて進められる
- GitHub など、目的に合ったサービスやサーバーを共有先として選べる

一方で、次のようなデメリットや注意点があります。

- 用語と基本操作を覚える必要があり、使い始めは現在の状態を把握しにくい
- 取り消す対象を間違えると、必要な変更を失うことがある
- 複数人が同じ場所を同時に編集すると、残す変更内容を調整する必要がある
- 大きなバイナリファイルを頻繁に更新する用途では、容量管理に工夫が必要になる

2 Git と GitHub の違い

Git と GitHub は同じものではありません。

- **Git** : PC 内でファイルの変更履歴を作成・管理するバージョン管理システム
- **GitHub** : Git リポジトリをネットワーク上で保管し、共有やレビューの機能を提供するサービス

1) Git はバックアップそのものではありません。PC が故障するとローカルの履歴も失う可能性があります。GitHub など別の場所にも履歴を送ることで、共有と保全に役立ちます。

Git の履歴やファイルの状態は目に見えにくいいため、この資料では随所でストップモーション制作²⁾にたとえます。作業中のファイルを撮影セット、コミットを記録済みのコマ、コミット履歴をコマが順に並ぶストップモーション動画として考えます。これは内部構造を厳密に表すものではなく、操作の流れをつかむための例えです。

この資料では、人形や小道具を少しずつ動かして撮ったコマをつなぎ、手元でストップモーション動画とその素材を管理する仕組みを Git、動画と素材をオンラインで共有し、相談やレビューを行う場所を GitHub と考えます。

手元の動画と素材 $\xrightarrow{\text{git push}}$ オンラインで共有・レビュー

2.1 リモートリポジトリの選択肢

手元のリポジトリから参照する、ネットワーク上や別ディレクトリのリポジトリをリモートリポジトリと呼びます。GitHub は代表的な選択肢ですが、唯一ではありません。

- 別のローカルディレクトリや共有サーバー上の Git リポジトリ
- GitHub
- GitLab などのホスティングサービス
- 組織が自分で運用するセルフホスト環境

Git とリモートの基本操作は共通ですが、アカウント、権限、課題管理、レビューなどの機能や名称はサービスごとに異なります。ストップモーションでいえば、リモートリポジトリは共有先に置いた動画とその素材、GitHub はそれらを基に相談やレビューもできる共同制作場所です。

3 手元環境への導入

3.1 Git をインストールする

授業や組織の指定がある場合は、そちらを優先します。インストール後はバージョンを確認します。

```
git --version
```

代表的な導入方法は次のとおりです。

- Windows: Git for Windows のインストーラー、または `winget install --id Git.Git -e --source winget`
- macOS: Xcode Command Line Tools、Git 公式インストーラー、または Homebrew の `brew install git`
- Debian / Ubuntu: `sudo apt install git`

2) ストップモーションは、人形などの対象を少しずつ動かして一コマずつ撮影し、連続して再生することで動いているように見せるアニメーション技法です。粘土製の人形を使うものはクレイアニメーションとも呼ばれます。[ピングー公式サイト](#)によると、**ピングー (Pingu)** は 1980 年に原型のテストフィルムが制作されたストップモーション作品です。

公式の導入案内は <https://git-scm.com/downloads> から確認できます。

3.2 Git の初期設定

コミットに記録する名前とメールアドレスを設定します。GitHub で使う場合は、GitHub アカウントで確認済みのメールアドレス、または GitHub の no-reply アドレスを使います。

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

設定を確認します。³⁾

```
git config --global --list
```

4 Git のローカル環境

Git で管理するファイルには、主に次の3つの状態があります。

- **変更済み (modified)**：前回のコミットから内容を変更したが、まだ次のコミットには含めていない
- **ステージ済み (staged)**：次のコミットへ含める内容を選び、スナップショットを準備した
- **コミット済み (committed)**：スナップショットをローカルリポジトリへ記録した

この3つの状態を扱うために、次の3つの領域を意識します。

1. **ワークツリー**：利用者がファイルを開いて編集するディレクトリと、その未コミットの内容
2. **インデックス (ステージングエリア)**：次のコミットへ含める内容の情報を保持する領域
3. **ローカルリポジトリ**：PC 内の `.git` に保存されたコミットと参照の集合

ワークツリー $\xrightarrow{\text{git add}}$ インデックス $\xrightarrow{\text{git commit}}$ ローカルリポジトリ

ステージとは、`git add` で指定内容のスナップショットをインデックスへ記録する操作です。

`git commit` は、インデックスの内容から新しいコミットを作成します。

この流れは、ストップモーションの一コマを次の順で記録する作業に似ています。

- 撮影セット上で、人形や小道具の配置を変更する
- 残したい配置になった時点で写真を撮る
- 撮影した写真へ日時とメモを付け、動画のタイムラインへ追加する

このとき、ワークツリーは撮影セット、`git add` は写真を撮る操作、インデックスは動画へ追加する前の写真に当たります。`git commit` は、その写真に作成者、日時、変更理由、直前のコミットとのつながりを加えて、ローカルリポジトリという手元の動画へ収める操作です。

3) 名前とメールアドレスはコミット履歴に記録されます。公開リポジトリへ送る可能性がある場合は、公開してよい情報が確認してください。

撮影セット $\xrightarrow{\text{git add}}$ 撮影済みの次のコマ $\xrightarrow{\text{git commit}}$ 手元のストップモーション動画

撮影後に配置を変えても写真が変わらないように、git add 後の編集はインデックスへ自動反映されません。反映するには、もう一度 git add します。なお、実際のインデックスは画像置き場ではなく、次のコミットを構成するファイル内容を保持する Git 固有の領域です。

5 リポジトリを作る

最初に、練習用リポジトリを置くディレクトリへ移動します。

```
cd PATH/TO/REPOSITORIES
```

5.1 管理するファイルを選ぶ

Git で管理するのは、別の人が clone したときに、成果物の内容を理解し、変更し、再現するために必要なファイルです。人が作成・編集し、変更理由を履歴として残したいファイルを管理します。一方、自動生成できるファイル、外部から再取得できるファイル、個人の環境だけで使うファイル、公開してはいけないファイルは原則として管理しません。

• 管理するもの

- ソースコード、テスト
- 文書、ライセンス
- 共同作業に必要な設定、ビルドスクリプト、依存関係のロックファイル
- 小容量の素材

• 原則として管理しないもの

- ビルド生成物
- 依存パッケージやキャッシュ
- 一時ファイル、ログ、エディターや OS 固有のファイル
- 認証情報
- 大容量のバイナリやデータセット

共同作業者にも共通する除外規則は .gitignore へ記述し、その .gitignore 自体はコミットします。個人だけのエディター設定などは、Git のグローバルな除外設定も利用できます。すでに追跡しているファイルは、後から .gitignore へ追加しただけでは追跡対象から外れません [1]。

認証トークン、秘密鍵、パスワードを含む .env などはコミットしません。必要な環境変数名だけを記した .env.example や設定例を管理し、実際の値は安全な認証情報管理の仕組みで共有します。

5.2 練習用ディレクトリを用意する

任意の作業場所で次を実行します。以降は git-practice 内で作業します。

```
mkdir git-practice
cd git-practice
git init -b main
git status
```

`git init` は現在のディレクトリを Git リポジトリにします。`-b main` は最初のブランチ名を `main` にします。

5.3 最初のコミットを作る

まだファイルがなくても、`--allow-empty` を付けると空のコミットを作れます。

```
git commit --allow-empty -m "Initial commit"
git log --oneline
```

`-m` の後ろには、何をしたコミットかを短く書きます。この最初のコミットが履歴の出発点になります。

5.4 README.md を作る

`README.md` は、リポジトリの目的、使い方、必要な環境を伝える案内文です。エディタで作成し、次の内容を保存します。

```
# Git practice

Git and GitHub practice repository.
```

5.5 .gitignore を作る

OS やエディタの自動生成ファイル、ビルド結果、ログなど、履歴に不要なファイルもあります。`.gitignore` にパターンを書くと、通常の追跡対象から除外できます。

エディタで `.gitignore` を作り、練習用として次の内容を保存します。

```
.DS_Store
*.log
build/
```

実際のプロジェクトでは、OS、言語、ツールに合わせて内容を決めます。GitHub のテンプレート集 <https://github.com/github/gitignore> も参考になります。

5.6 README.md と.gitignore をコミットする

先ほど作成した `README.md` と `.gitignore` を、ファイルを含む最初のコミットとして記録します。この2ファイルはまだ未追跡です。未追跡ファイルの内容は通常の `git diff` には表示されないため、`git status` で対象を確認し、`git add` の後に `git diff --staged` でコミット予定の

内容を確認します。

```
git status
git add README.md .gitignore
git status
git diff --staged
git commit -m "Add README and ignore rules"
git log --oneline
```

追跡済みファイルを変更した場合は、status → diff → add → status → diff --staged → commit → log の順に確認します。今回のような未追跡ファイルではステージ前の git diff に内容が出ないため、最初の diff を省いています。⁴⁾

- git status：現在のブランチ、追跡状態、ステージ状態を表示する
- git diff：まだステージしていない変更の差分を表示する
- git add：実行時点の内容を、次のコミットに含めるスナップショットとしてインデックスへ記録する
- git diff --staged：コミット予定の差分を確認する
- git commit：ステージした変更を履歴に記録する
- git log：コミット履歴を確認する

git add のあとでファイルを再び編集すると、インデックスには git add 実行時点の内容が残り、その後の編集はワークツリーだけにあります。その編集も次のコミットへ含めるには、もう一度 git add を実行します。

6 直線的なコミットを作る

```
cd PATH/TO/git-practice
```

6.1 README.md を追加で編集してコミットする

README.md の末尾へ次の説明を追加して保存します。

```
## Purpose

Practice the basic Git workflow.
```

追加した内容を確認し、同じ流れで新しいコミットを作ります。

4) [Pro Git 2.2「変更内容のリポジトリへの記録」](#)では、git diff はステージ前の変更、git diff --staged はコミット対象としてステージした変更の確認に使うと説明されています。

```
git status
git diff
git add README.md
git status
git diff --staged
git commit -m "Explain practice repository"
git log --oneline --decorate
```

新しいコミットは直前のコミットにつながり、直線的な履歴になります。⁵⁾

これは、少しずつ配置を変えて撮ったコマが順番に並ぶ様子と同じです。各コマが親コミットを参照してつながる履歴に当たり、Git でも作業の節目ごとにコミットすると変更の過程をたどりやすくなります。

6.2 参考：変更の一部だけをステージする

この項目は、`git add -p` と `git add -i` の機能を知るための紹介です。ここに示すコマンドは、前項から続く練習用リポジトリでは実行しません。

一つのファイルへ複数の目的の変更を加えた場合、ファイル全体を一度にステージせず、次のコミットへ含める部分だけを選べます。`git add -p` の `-p` は `--patch` の短縮形です。

ハンク (hunk) とは、差分の中で近接する変更行をまとめた選択単位です。⁶⁾ 次のコマンドは、`README.md` のハンクを順に表示し、選んだ部分だけをステージします。

```
git diff
git add -p README.md
git diff --staged
git diff
git status
```

主な回答は次のとおりです。

- `y` : 表示中のハンクをステージする
- `n` : 表示中のハンクをステージせず、次へ進む
- `s` : 分割できるハンクを、さらに小さなハンクへ分ける
- `q` : 終了し、まだ判断していないハンクはステージしない

操作後は、`git diff --staged` で選んだ変更を、`git diff` で残した変更を確認します。同じファイルの一部だけをステージすると、`git status` では、そのファイルがステージ済みと未ステージの両方に表示されることがあります。`git commit` に含まれるのはステージ済みの部分だけです。

`git add -i` の `-i` は `--interactive` の短縮形です。対話メニューから、ステージ対象の更新、

5) `git add .` は現在位置以下の複数の変更をまとめてステージします。便利ですが、意図しないファイルを含めないうように、先に `git status` と `git diff` を確認します。

6) ハンクは変更の目的ではなく、差分中の位置によって分かります。`s` で分割できない場合は、無理に選ばず、ファイルの編集内容を整理してからもう一度 `git add -p` を実行します。

取り消し、未追跡ファイルの追加、パッチ選択、差分確認などを選びます。

```
git add -i
```

`git add -p` は、この対話モードの最初のメニューを省き、直接パッチ選択へ入る操作です。変更の一部を選ぶだけなら `-p`、複数のステージ操作を一つの対話メニューから選ぶなら `-i` を使います。

6.3 参考：VS Code の Copilot でコミットメッセージ案を作る

VS Code では、GitHub Copilot を使い、ステージ済みの変更を基にコミットメッセージ案を生成できます。この機能が補助するのはメッセージの作成です。ステージする内容の選択と、コミットを実行する判断は利用者が行います。

利用するには、GitHub Copilot を利用できる GitHub アカウントで VS Code へサインインし、Git リポジトリのフォルダを開きます。Copilot をまだ有効にしていない場合は、ステータスバーの Copilot アイコンから **Use AI Features** を選び、画面に従ってサインインします。

1. VS Code の **Source Control** を開く
2. **Changes** にあるファイルの差分を確認する
3. 次のコミットへ含めるファイルの **+** を選び、**Staged Changes** へ移す
4. ステージ済み差分をもう一度確認する
5. コミットメッセージ入力欄の **きらめきアイコン** を選ぶ
6. 生成された **コミットメッセージ案** を差分と照合し、必要なら修正する
7. 内容に問題がなければ **Commit** を選ぶ

コミットメッセージ案とは、ステージ済みの変更を基に Copilot が生成し、コミット前に利用者が確認・修正する文章です。生成されるのはメッセージ案だけで、ステージ済みのファイル内容は変わりません。

複数の目的を一度にステージすると、生成案も複数の変更理由をまとめた曖昧な内容になりやすくなります。先に差分を目的ごとの小さな単位へ分け、メッセージには「何を変えたか」だけでなく「なぜ変えたか」が伝わる表現を残します。

生成案には、変更理由の取り違いや重要な変更の見落としがあり得ます。必ずステージ済み差分と照合し、認証情報や秘密情報が含まれていないことも確認してからコミットしてください。

7 GitHub の準備と認証

GitHub アカウントの作成と GitHub CLI の認証は、別資料「GitHub アカウント作成手順」を参照してください。別資料のリポジトリルートからのパスは `doc/github-account.pdf` です。組織から管理アカウントを支給されている場合は、別資料よりも組織の案内を優先します。

次節へ進む前に、GitHub CLI を <https://cli.github.com/> の手順で導入し、バージョンと認証状態を確認します。

```
gh --version
gh auth status
```

`gh auth status` で認証済みの GitHub アカウントが表示されれば準備完了です。認証されていない場合は、別資料の手順に従って `gh auth login` を実行します。

8 GitHub へ push する

```
cd PATH/TO/git-practice
```

GitHub へ push すると、ローカルのコミットと履歴を共有でき、別の端末への作業引き継ぎ、共同作業者とのレビュー、成果公開に利用できます。

次の 2 つは同じ目的の代替手段なので、どちらか一方を実行します。

8.1 GitHub の Web UI を使う

GitHub の New repository から `git-practice` を作成します。すでに手元にコミットがあるため、README、`.gitignore`、LICENSE は GitHub 側で追加せず、空のリポジトリにします。

作成後に表示される HTTPS URL を使います。YOUR-NAME は自分の GitHub ユーザー名へ置き換えます。

```
https://github.com/YOUR-NAME/git-practice.git
```

作成後に表示された URL を origin として設定します。

```
git remote add origin https://github.com/YOUR-NAME/git-practice.git
```

8.1.1 リモートを確認する

origin が設定されたことを確認します。

```
git remote -v
```

origin はリモートにつける慣例的な短い名前です。特別な機能を持つ予約語ではありません。

8.1.2 コミットを送る

次のコマンドでコミットを送ります。

```
git push -u origin main
```

push とは、ローカルブランチのコミットをリモートブランチへ転送する操作です。-u は、

main の上流ブランチを設定します。2回目以降は、多くの場合 `git push` だけで送れます。⁷⁾

これは、手元の動画で確定したコマと、そのコマで使う素材を共有先へ送る操作に当たります。撮影セットで調整中の配置や、動画へ追加していない写真と素材は送られません。

GitHub のリポジトリページを更新し、README とコミット履歴が表示されることを確認します。

8.2 GitHub CLI (gh) を使う

git-practice ディレクトリで次のコマンドを実行します。

```
gh repo create
```

表示される質問に沿って、既存のローカルリポジトリから作成する方法、リポジトリ名 git-practice、公開範囲を順に選びます。リモートの追加を確認されたら追加し、名前は origin とします。続けてコミットの push を確認されたら push します。公開範囲は、公開してよい内容だけで構成されていると確認できなければ非公開を選びます [2]。⁸⁾

GitHub のリポジトリページを開き、README とコミット履歴が表示されることを確認します。

9 リモートリポジトリを clone する

最初に、clone 先を置く親ディレクトリへ移動します。

```
cd PATH/TO/REPOSITORIES
```

clone は、リモートリポジトリのファイルと履歴を新しいディレクトリへ複製し、通常は origin も自動設定します。

撮影なら、共有先にある動画とその素材を一式複製し、続きを制作できる手元の撮影セットを用意する操作に当たります。

ここでは、公開リポジトリ [kska6/introduction-latex](https://github.com/kska6/introduction-latex) を例に使います。次の 2 つは同じ目的の代替手段なので、どちらか一方を実行します。

9.1 git clone を使う

```
git clone https://github.com/kska6/introduction-latex.git
cd introduction-latex
git remote -v
git log --oneline
```

7) push が送るのはコミットです。未コミットの変更は送られません。先に `git status` と `git log --oneline` で確認します。

8) 質問への回答をオプションで指定する場合は、`gh repo create git-practice --private --source=. --remote=origin --push` と実行できます。

9.2 gh repo clone を使う

GitHub CLI で認証済みなら、OWNER/REPOSITORY の形式も使えます。⁹⁾

```
gh repo clone kska6/introduction-latex introduction-latex-gh
```

共同作業では、他の人がリモートへ追加したコミットを `git pull` で取得・統合します。作業前に `git pull`、作業後に `git push` という流れが一例ですが、チームのルールを優先します。

10 参考：作業中の変更を整理する

本節は、作業中の変更を整理するための機能紹介です。ここに示すコマンド例は、前節までの練習用リポジトリでは実行しません。

10.1 変更を取り消す

取り消す対象によって操作は異なります。`restore` は未コミットの内容、`revert` は履歴を残して打ち消す変更、`reset` は履歴の先端を操作します。実行前に `git status` と `git log --oneline` を確認してください。

10.1.1 ワークツリーの未コミット変更を戻す

`README.md` を編集したあと、その変更を捨てて最後のコミット時点に戻す場合は次を使います。

```
git diff
git restore README.md
git status
```

`git restore README.md` は、ワークツリーの `README.md` をインデックスの内容へ戻します。まだステージしていなければ、通常はインデックスと直前のコミットの内容が一致しています。

撮影でいえば、動画と撮影済みの写真は変えず、撮影セットだけを写真の配置へ戻す操作です。つまり、インデックスを基準にワークツリーを戻します。

`git restore README.md` で捨てた未コミット変更は、通常 Git から復元できません。必要なら事前にコピーします。

10.1.2 ステージを取り消す

ファイルの編集内容は残したまま、`git add` だけを取り消します。

```
git restore --staged README.md
git status
```

9) すでに Git 管理しているディレクトリの中へ、同じリポジトリを重ねて clone しないでください。clone は原則として、そのリポジトリを置きたい親ディレクトリで実行します。

`git restore --staged README.md` は、ワークツリーの編集内容を残したまま、インデックスの `README.md` を `HEAD` の内容へ戻します。

撮影では、セットはそのままに、動画へ追加する前の写真だけを破棄する操作に当たります。コミット履歴とワークツリーの編集内容は変わりません。

10.1.3 コミットした変更を打ち消す

すでにコミットした変更、特に GitHub へ共有済みの変更を取り消す場合は、`git revert` を使います。`revert` は指定したコミットの変更を逆向きに適用し、その結果を新しいコミットとして記録します。元のコミットを履歴から消さないため、共有された履歴を保ったまま変更を打ち消せます。

ストップモーションなら、人形が一步進んだコマを削除せず、元の位置へ戻るコマを後から加えます。「進んでから戻った」という経緯も履歴に残ります。

直前のコミットを打ち消す場合は次のようにします。

```
git log --oneline
git revert HEAD
git log --oneline
git status
```

コミットメッセージを編集する画面が開いた場合は、内容を確認して保存し、エディタを終了します。直前以外のコミットを指定する場合は、`git revert COMMIT-ID` のようにコミット ID を指定します。

10.1.4 直前のコミットをやり直す

まだ GitHub に送っていない直前のコミットを取り消し、変更をステージしたまま残す例です。

`git reset` は、ブランチが指すコミットを過去へ移し、指定した方式に応じてインデックスとワークツリーを更新します。打ち消し用の新しいコミットは作りません。

動画編集なら、タイムラインの末尾を過去のコマへ移し、それ以降を作品の続きから外す操作に当たります。写真や撮影セットを残すかどうか、`reset` のオプションによって変わります。

```
git reset --soft HEAD~1
git status
```

`--soft` では、ブランチの指すコミットだけを戻し、取り消したコミットの内容をインデックスに残します。

ストップモーションでいえば、タイムラインから外したコマを動画へ戻せる写真として残す状態です。

変更をワークツリーに残し、ステージだけ外す場合は次を使います。

```
git reset HEAD~1
git status
```

モードオプションを指定しない `git reset` では、ブランチの指すコミットを戻してステージを外しますが、ワークツリーの変更は残します。

ストップモーションでいえば、タイムラインから外したコマは破棄しても、撮影セットの配置は残す状態です。

`git reset --hard` はコミットと未コミット変更をまとめて失う可能性があります。また、共有済みのコミットに対する `reset` は他の人の履歴と食い違う原因になります。初心者演習では使いません。共有済みの変更を打ち消すときは、前節の `git revert` を使います。

10.2 変更を一時退避する

`stash` とは、作業中の変更をワークツリー外へ一時退避した記録です。別の対応へ切り替えたいとき、`git stash` で追跡済みファイルの変更を退避できます。

これは、撮影途中の配置を控えてセットを片付け、後で元へ戻す作業に似ています。退避が `git stash`、復元が `git stash pop` または `git stash apply` に当たります。

```
git status
git stash push -m "WIP: edit README"
git status
git stash list
git stash pop
git status
```

`stash pop` は退避内容を戻し、正常に適用できれば一覧から削除します。退避を残したまま適用する場合は `git stash apply` を使います。¹⁰⁾

11 ブランチを使う

```
cd PATH/TO/git-practice
```

ブランチとは、履歴の先端にあるコミットを指す、移動可能な名前付き参照です。コミットを追加すると新しい先端へ進むため、元の履歴を保ちながら作業を分けられます。

ストップモーションでも、同じコマから別の演出を撮れば、コマの並びが分岐します。各系列の末端を示す名前付きの札がブランチに当たります。ブランチ自体は系列全体のコピーではなく、一つのコミットを指します。

10) 新規作成した未追跡ファイルも退避する場合は `git stash -u` を使います。`stash` は長期保存場所ではありません。作業の節目はコミットとして残します。

11.1 作成と切り替え

```
git branch
git branch add-greeting
git switch add-greeting
git branch
```

作成と切り替えを一度に行う場合は次のようにします。

```
git switch -c add-greeting
```

同名のブランチがすでにある場合は失敗するため、そのときは `git switch add-greeting` を使います。

`git switch` は、作業対象とするブランチを切り替え、ワークツリーをそのブランチの状態に合わせます。

撮影では、続きを撮る系列を選び、セットを最後のコマに合わせます。系列の選択がブランチの切り替え、セットの調整がワークツリーの更新に当たります。

ブランチを切り替えたら `README.md` に挨拶を追加し、コミットします。

```
git status
git add README.md
git commit -m "Add greeting"
git log --oneline --decorate --graph --all
```

11.2 ブランチをマージする

`add-greeting` の変更を `main` に取り込みます。取り込み先へ先に切り替えることが重要です。

```
git switch main
git merge add-greeting
git log --oneline --decorate --graph --all
git push origin main
```

履歴が分岐していなければ、`main` の位置だけが前へ進む `fast-forward` になることがあります。双方に別のコミットがある場合は、統合したことを表すマージコミットが作られます。

撮影なら、一方で背景を夜景にし、もう一方で人形の右手を上げた場合、両方を一つの場面へ合わせます。これが、別ブランチの変更を現在のブランチへ取り込む `git merge` に当たります。

同じ行を別々のブランチで変更すると、Git が残す内容を自動で選べず、**コンフリクト**になることがあります。ストップモーションでいえば、同じ人形の右手を一方では上げ、もう一方では下げた状態です。制作者が姿勢を決めるように、ファイルを開いて残す内容を決め、競合を示す印を削除してから `git add` と `git commit` で結果を記録します。マージ自体を中止する場合は `git merge --abort` を使います。

12 参考：履歴の節目にタグを付ける

本節は、履歴の節目へ名前を付けて共有するタグの機能紹介です。ここに示すコマンド例は、前節までの練習用リポジトリでは実行しません。

タグとは、リリースなど履歴上の節目として後から参照できるよう、特定のコミットへ付ける名前です。新しいコミットに応じて先端へ進むブランチとは異なり、タグは作成後も同じコミットを指し続けます。

ストップモーションなら、制作中の系列の末端を示す札がブランチで、完成版として公開するコマへ付けたラベルがタグに当たります。Git では、v1.0.0 のような名前がよく使われますが、名前の規則はプロジェクトごとに決めます。

12.1 注釈付きタグを作成して確認する

注釈付きタグとは、対象に加えて作成者、作成日時、メッセージを保存するタグです。リリースなど共有する節目には、注釈付きタグを使うと、後から作成理由を確認できます。

現在のコミットへ v1.0.0 という注釈付きタグを付ける例は次のとおりです。

```
git status
git log --oneline -1
git tag -a v1.0.0 -m "Release v1.0.0"
git tag
git show v1.0.0
```

git tag はローカルリポジトリにあるタグの一覧を表示し、git show v1.0.0 はタグの情報と、そのタグが指すコミットを表示します。特定の過去のコミットへ付ける場合は、git tag -a TAG-NAME COMMIT-ID -m "MESSAGE" のようにコミット ID を指定します。¹¹⁾

12.2 タグを GitHub へ送る

通常の git push では、ローカルで作成したタグは自動的に送られません。共有するタグの名前を明示して送ります。

```
git push origin v1.0.0
```

タグを送った場合は、GitHub のリポジトリページで表示されることを確認します。複数のタグをまとめて送る git push origin --tags もありますが、意図しないタグまで公開しないよう、送るタグを一つずつ指定する方法が安全です。¹²⁾

11) -a とメッセージを付けずに作るタグは**軽量タグ**です。軽量タグは対象を指す名前だけを保持します。この資料では、リリースの節目を明確に残すため注釈付きタグを使います。

12) 共有済みのタグを別のコミットへ付け直すと、利用者ごとに同じ名前が異なるコミットを指す原因になります。誤りを直す場合は、チームで確認し、通常は別のタグ名を付けます。

13 応用：Issue と Pull Request

```
cd PATH/TO/git-practice
```

13.1 Issue

Issue とは、GitHub 上で問題や提案、質問を記録し、完了条件や議論をまとめる機能です。目的、理由、完了条件を書くと、作業の起点になります。

13.2 Pull Request

Pull Request (PR) は、あるブランチの変更を別のブランチへ取り込む提案です。差分を基に説明、相談、レビューを行い、合意後にマージします。

同様に、別の撮影系列で作った演出を本編へ採用する提案が Pull Request、統合前の確認が差分レビューに当たります。

小さな共同作業の例は次のとおりです。

1. Issue で目的を確認する
2. main から作業ブランチを作る
3. 変更し、内容ごとにコミットする
4. ブランチを GitHub へ push する
5. PR を作り、目的と変更内容、確認方法を書く
6. レビューを受け、必要なら追加コミットする
7. 承認後に main へマージする

```
git switch -c fix-typo
# Edit files, then review the changes.
git status
git diff
git add README.md
git commit -m "Fix README typo"
git push -u origin fix-typo
gh pr create --fill
```

コントリビューションとは、Issue の報告、文書やコードの変更、レビューなどによるプロジェクトへの貢献です。公開プロジェクトでは、README、CONTRIBUTING.md、行動規範、ライセンスを先に確認します。

14 応用：worktree で作業場所を分ける

```
cd PATH/TO/git-practice
```

複数の作業を同じディレクトリで同時に進めると、一方の未コミット変更がもう一方へ混ざっておそれがあります。git worktree を使うと、同じリポジトリに別の作業場所を追加し、変更を分けて進められます。

14.1 追加の worktree を作る

前半で説明したワークツリーは、一つの作業場所にある編集対象のファイル群です。git worktree は、同じリポジトリへ複数の作業場所を関連付けて管理するコマンドです。

リンクされた worktree とは、同じリポジトリのコミットや参照を共有しながら、別のブランチを独立したディレクトリで扱う追加の作業場所です。clone が別のリポジトリ一式を作るのに対し、リンクされた worktree は元のリポジトリと履歴やブランチを共有します。

main から docs/update-guide ブランチを作り、一つ上の階層へ worktree を追加する例は次のとおりです。

```
git status
git worktree add -b docs/update-guide ../git-practice-docs main
git worktree list
```

追加したディレクトリは固有のワークツリー、インデックス、HEAD を持つため、元の作業場所にある未コミット変更と混ざりません。¹³⁾

14.2 活用例：LLM Agent の作業場所を分ける

Git を使うと、変更の差分や理由を履歴として残し、ブランチで作業を分け、他の人と共有できます。一方で、状態の確認、目的に合うコマンドやオプションの選択、コミットメッセージの作成は繰り返し発生します。この資料では、履歴管理のために繰り返すこれらの作業を **Git 操作の負担**と呼びます。

LLM（大規模言語モデル）は、差分からコミットメッセージ案を生成したり、目的に合うコマンドを説明したりできます。さらに LLM Agent は、リポジトリの状態を調べ、許可された範囲でコマンドを実行し、その結果を利用者へ返せます。状態確認と Git コマンドの実行を Agent へ任せ、結果を利用者が確認する使い方を **LLM Agent への操作委任**と呼びます。

これらを利用すると、Git が持つ履歴管理や共同作業の利点を得ながら、すべてのコマンドを利用者自身が入力する必要はなくなります。利用者の役割は、個々の操作を手作業で行うことから、依頼する範囲を決め、差分と実行結果を確認することへ移ります。

ただし、LLM や LLM Agent を使っても、ワークツリー、インデックス、コミット、ブランチの関係、アクセス権限、コンフリクトなど、Git の制約自体は変わりません。現在の状態で何を実行できるか、どの操作が制限されるか、履歴やファイルへどのような影響があるかを判断するために、Git の基本的な仕組みを理解しておく必要があります。

Agent がコマンドを実行できることと、その操作が目的に合っていることは別です。対象のリポジトリとブランチ、コミットへ含める差分、共有済み履歴への影響を確認してから実行を承認

13) Git は、同じブランチを複数の worktree で同時にチェックアウトすることを通常は許可しません。作業場所ごとに別のブランチを使います。

します。

LLM Agent へリポジトリの調査、編集、検証を依頼するとき、専用の worktree を作業場所として渡す使い方があります。利用者の作業中の変更と Agent の変更を分けたまま、結果の差分を確認できます。

```
cd ../git-practice-docs
git status
# The LLM Agent works in this directory.
git diff
```

Agent の作業後は、差分と検証結果を利用者が確認し、必要な変更だけを採用します。ただし、別のブランチで同じ行を変更すると、統合時にコンフリクトが起こることがあります。

14.3 worktree を片付ける

追加した worktree が不要になったら、必要な変更がコミットされ、統合済みであることを確認してから、元の作業場所で削除します。

```
cd ../git-practice
git worktree list
git worktree remove ../git-practice-docs
git branch -d docs/update-guide
```

git worktree remove は、追加した作業ディレクトリを Git の管理情報とともに削除します。未コミット変更がある worktree は通常削除を拒否します。Finder や rm だけでディレクトリを消さず、このコマンドを使います。git branch -d も、ブランチが統合済みであることを確認してから実行します。

15 応用：リポジトリ構成を設計する

```
cd PATH/TO/git-practice
```

15.1 基本は 1 プロジェクト 1 リポジトリ

基本的には、一つのプロジェクトを一つの Git リポジトリで管理します。1 プロジェクト 1 リポジトリとは、一つの成果物を構成するソースコード、設定、テスト、文書と、それらの履歴を、一つのリポジトリへまとめる方針です。

一つにまとめると、プロジェクト全体の変更を同じコミットや Pull Request で確認でき、clone やセットアップの手順も単純になります。初心者のうちは、明確に分ける理由がなければ、この構成から始めます。

15.2 リポジトリの容量と管理対象

1 プロジェクト 1 リポジトリは、関係するすべてのファイルを無制限にコミットする方針ではありません。大容量バイナリ、ビルド生成物、依存キャッシュ、一時ファイルなどを繰り返し履歴へ入れると、clone、fetch、保存、レビューにかかる負担が増えます。.gitignore や成果物の保管先を使い分け、Git で履歴を残す必要があるファイルを選びます。ファイルを後から削除しても過去のコミットには残るため、リポジトリを巨大にしすぎないことが重要です。

GitHub を利用する場合は、リポジトリの容量を定期的に確認します。¹⁴⁾ 容量の目安を超えそうなときは、まず大容量バイナリや生成物が履歴に含まれていないかを確認し、Git LFS、Release、外部ストレージなどへの移動を検討します。そのうえで、構成要素を独立して管理できる場合にリポジトリの分割を検討します。

15.3 複数リポジトリをサブモジュールで組み合わせる

大規模なプロジェクトでは、構成要素ごとにリリース周期、アクセス権限、再利用先、担当範囲が異なることがあります。このように履歴や公開範囲を独立して管理する必要がある場合は、リポジトリを複数に分ける方針もあります。単にファイル数が多いという理由だけで分けるのではなく、独立して管理する必要があるかで判断します。

サブモジュールとは、親リポジトリが、指定パスに置く別リポジトリの URL と使用する特定コミットを記録する仕組みです。Git の公式資料では、サブモジュールを含む側を `superproject` と呼びます。この資料では**親リポジトリ**と表記します。

既存の部品リポジトリを `modules/component` へ追加する例は次のとおりです。

```
git submodule add https://github.com/OWNER/component.git modules/component
git status
git commit -m "Add component as submodule"
```

親リポジトリには、サブモジュールの URL を含む `.gitmodules` と、使用する子リポジトリの特定コミットが記録されます。子リポジトリの最新版へは自動で追従しません。子リポジトリを更新した後は、親リポジトリでも使用するコミットをステージしてコミットします。

サブモジュールを含めて clone する場合は、次のようにします。

```
git clone --recurse-submodules REPOSITORY-URL
```

通常の clone を実行した後にサブモジュールを取得する場合は、親リポジトリ内で `git submodule update --init --recursive` を実行します。¹⁵⁾

14) 目安として、テキストのみの場合は数 MB から数十 MB に収まることが多く、100MB を超えたら不要な生成物や大きな履歴がないか点検します。画像が入る場合は、1 枚 5MB の画像 100 枚で約 500MB となり、数百 MB から 1GB へ達しやすくなります。GitHub は 1GB 未満を理想とし、5GB 未満に保つことを強く推奨しています。これは Git 自体の上限ではありません [3]。

15) サブモジュールでは、親と子のそれぞれでブランチ、コミット、push を管理します。clone や更新の手順も増えるため、独立管理の必要性がないプロジェクトでは、1 プロジェクト 1 リポジトリの方が単純です。

16 応用：Git の仕組み

```
cd PATH/TO/git-practice
```

ここからは、基本操作を理解したあとに読む内容です。

16.1 コミットはスナップショットをつなぐ

Git はコミット時点のファイル構成をスナップショットとして扱います。各コミットは親コミットを参照し、そのつながりが履歴になります。

16.2 保存内容から識別子を計算する

ハッシュ関数とは、任意長の入力から一定長の値を計算する仕組みです。Git は保存内容からオブジェクト ID を計算し、保存単位を識別します。

Git オブジェクトとは、Git 内部の保存単位です。主な種類は、ファイル内容を格納する**Blob**、ファイル名やディレクトリ構成を記録する**ツリー**、スナップショット、作成者、メッセージ、親コミットを記録する**コミットオブジェクト**です。

内容が変わればオブジェクト ID も変わります。計算方法を覚える必要はなく、「各コミットには履歴上の住所のような ID がある」と理解すれば十分です。

```
git log --oneline
git show COMMIT-ID
```

ブランチは、履歴の先端にあるコミットを指します。**HEAD** は通常、現在チェックアウトしているブランチを指す参照ですが、特定のコミットを直接指す場合もあります。

Git の仕組みをさらに学ぶ読み物として、[Pro Git 日本語版](#)を参照してください。特に第 10 章「Git の内側」では、Git オブジェクトや参照の仕組みを詳しく学べます。

17 公式資料

- [1]Git Project. *gitignore Documentation*. URL: <https://git-scm.com/docs/gitignore>.
- [2]GitHub CLI Manual. *gh repo create*. URL: https://cli.github.com/manual/gh_repo_create.
- [3]GitHub Docs. *About large files on GitHub*. URL: <https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github>.
- [4]Git Project. *Git Reference*. URL: <https://git-scm.com/docs>.
- [5]Scott Chacon and Ben Straub. *Pro Git: バージョン管理に関して*. URL: <https://git-scm.com/book/ja/v2>.
- [6]Git Project. *Git Cheat Sheet*. URL: <https://git-scm.com/cheat-sheet.pdf>.
- [7]Git Project. *git-add Documentation*. URL: <https://git-scm.com/docs/git-add>.
- [8]Git Project. *git-tag Documentation*. URL: <https://git-scm.com/docs/git-tag>.

-
- [9]Git Project. *git-worktree Documentation*. URL: <https://git-scm.com/docs/git-worktree>.
 - [10]Git Project. *git-submodule Documentation*. URL: <https://git-scm.com/docs/git-submodule>.
 - [11]Git Project. *git-submodules Documentation*. URL: <https://git-scm.com/docs/git-submodules>.
 - [12]Scott Chacon and Ben Straub. *Pro Git: タグ*. URL: <https://git-scm.com/book/ja/v2>.
 - [13]OpenAI. *Codex: Worktrees*. URL: <https://learn.chatgpt.com/docs/environments/git-worktrees>.
 - [14]OpenAI. *Codex: Best practices*. URL: <https://learn.chatgpt.com/guides/best-practices>.
 - [15]Microsoft. *Visual Studio Code: Staging and committing changes*. URL: <https://code.visualstudio.com/docs/sourcecontrol/staging-commits>.
 - [16]Microsoft. *Visual Studio Code: Set up GitHub Copilot*. URL: <https://code.visualstudio.com/docs/setup/copilot>.
 - [17]GitHub Docs. *GitHub Docs*. URL: <https://docs.github.com/ja>.
 - [18]GitHub CLI Manual. *GitHub CLI Manual*. URL: <https://cli.github.com/manual/>.